

pamica: GPU-accelerated Adaptive Mixture Independent Component Analysis in Python with Fortran parity

Seyed Yahya Shirazi¹, Arnaud Delorme^{1,2}, and Scott Makeig¹

¹ Swartz Center for Computational Neuroscience, Institute for Neural Computation, University of California San Diego, USA ² Centre de Recherche Cerveau et Cognition (CerCo), CNRS, University of Toulouse, France ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Independent Component Analysis (ICA) is a standard method for separating electroencephalography (EEG) and electromyography (EMG) recordings into maximally independent sources, which isolates brain, muscle, and artifact activity for downstream analysis. Adaptive Mixture ICA (AMICA) generalizes single-model ICA to a mixture of ICA models with adaptive source densities, and produces the most dipolar (and thus most physiologically interpretable) component decompositions of EEG among the widely used algorithms benchmarked by Delorme et al. (2012). Its reference implementation is a Fortran program parallelized with the Message Passing Interface (MPI) and distributed as a compiled binary driven from MATLAB/EEGLAB, which is difficult to install, runs only on the CPU, and is not usable from a Python scientific workflow.

pamica is a Python implementation of AMICA that reproduces the reference Fortran results within numerical tolerance while running on the CPU, NVIDIA GPUs (CUDA), and Apple GPUs (Apple's MLX array framework (Hannun et al., 2023)). It is a complete NumPy/PyTorch reimplementation, not a wrapper around the Fortran binary, built on PyTorch (Paszke et al., 2019), NumPy (Harris et al., 2020), and SciPy (Virtanen et al., 2020), and exposes a scikit-learn-style estimator under a BSD-3-Clause license. In double precision it reproduces the reference score-function algebra to machine precision; it also runs in single precision (float32), which the CPU-only binary does not offer and which is required to use Apple GPUs, which have no float64 and host the fastest backend (MLX). pamica writes output in the binary format that EEGLAB's AMICA loader reads: a single-model output file is byte-identical in layout to a native AMICA file, and multi-model output round-trips through the same loader. Correctness is defined as parity with the Fortran reference for the single-model case and, because multi-model AMICA is not partition-identifiable, as a similar distribution of solutions for the multi-model case; both are validated on real EEG against the reference binary. The software is at <https://github.com/scen/pAMICA> (archived at doi:10.5281/zenodo.21312148).

Statement of need

AMICA also supports equivalent-dipole source localization and automated component classification (Pion-Tonachini et al., 2019). As neuroimaging analysis moves toward Python, for example MNE-Python (Gramfort et al., 2013), its MATLAB-only Fortran implementation is an increasing obstacle: an AMICA that runs natively in Python and on a GPU, and that is validated to reproduce the Fortran reference numerically, is needed for modern pipelines.

General-purpose Python ICA implementations do not fill this gap. scikit-learn and MNE-

Python provide FastICA (Hyvärinen & Oja, 2000) and Infomax (Bell & Sejnowski, 1995; Lee et al., 1999), and Picard (Ablin et al., 2018) provides a faster maximum-likelihood ICA, but none implement AMICA's mixture of models, adaptive generalized-Gaussian source densities, or Newton updates, and so they do not reproduce AMICA decompositions. pamica targets EEG/EMG analysts who want AMICA-quality decompositions inside a Python pipeline, users of GPU hardware who want faster runs than the CPU-only binary, and methodologists who need a transparent reference implementation to build on.

Implementation and validation

pamica provides a natural-gradient (Amari, 1998) expectation-maximization backend that ports the full AMICA algorithm: exact-EM mixture updates, a positive-definite Newton step (Palmer et al., 2008), symmetric zero-phase-component-analysis (ZCA) sphering, the five source-density families of the reference (generalized Gaussian, Gaussian, logistic, sub-Gaussian, and the extended-Infomax kurtosis switcher), a mixture of ICA models, and component sharing across models. It also computes mutual information reduction (MIR) and pairwise mutual information (PMI), the separation-quality metrics used to benchmark ICA algorithms (Delorme et al., 2012).

pamica's conformity with the reference binary is measured with two complementary metrics: Hungarian-matched component correlation and the Amari distance (Amari et al., 1996), a permutation- and scale-invariant unmixing-matrix metric that needs no assignment step. Both implementations were run for AMICA's usual 2000 iterations with Newton off (`do_newton=0`) and otherwise-default parameters (settings transcribed between pamica's JSON and Fortran's native text format). Newton is disabled here to isolate the algorithm from initialization: with Newton on, independently seeded runs can settle a few under-determined components in different, equally likely optima, though a matched initialization recovers agreement (documentation). The single-model headline (Table 1) uses an external recording (OpenNeuro ds002718, 70 channels, $k \approx 153$ frames per squared channel count), well past the ~ 60 threshold where cross-backend agreement plateaus; the bundled 32-channel sample below ($k \approx 30$) gives a consistent Amari distance. Score functions and per-block sufficient statistics are exact to floating-point resolution against the literal Fortran expressions on the bundled sample. A mixture of ICA models is not partition-identifiable, so exact partition parity is the wrong bar for the multi-model case; it is instead assessed by whether the two implementations sample a similar distribution of solutions, across ensembles of 20 runs each (Figure 1). A permutation test finds no evidence that cross-implementation agreement is worse than Fortran's own run-to-run agreement. Multi-model log-likelihood distributions still differ slightly at a matched iteration budget (pamica needs about twice as many iterations to reach Fortran's mean), so full-likelihood similarity is not yet claimed.

Table 1: Single-model parity (external ds002718, $k \approx 153$; Amari and score-function checks on the bundled sample) and multi-model distributional similarity (bundled sample) of pamica with the Fortran reference. All values are means (sd shown where relevant) over matched components (single-model) or within/cross-implementation pairs (190/400, multi-model).

Regime	Metric	Result (mean; correlation / Amari distance)
Single-model	Log-likelihood gap to Fortran (mean per-sample-channel LL, -3.6993 , $k \approx 153$)	within ~ 0.0005
Single-model	Conformity with Fortran	0.998 ($k \approx 153$) / 0.006 (bundled, $k \approx 30$)

Regime	Metric	Result (mean; correlation / Amari distance)
Single-model	Score functions (non-default families) and sufficient statistics	exact to floating-point resolution ($\sim 10^{-15}$)
Multi-model	Single-run magnitude (pamica-Fortran; Fortran-Fortran)	0.65; 0.64 (sd 0.05) / 0.163; 0.174 (sd 0.02)
Multi-model	Ensemble similarity, 20 runs each: mean difference, between — within-Fortran (permutation p)	+0.011 ($p = 0.96$) / -0.011 ($p > 0.999$)

Multi-model AMICA (n_models=2): pyAMICA vs Fortran ensembles, real sample EEG

Dashed vertical lines mark each distribution's mean.

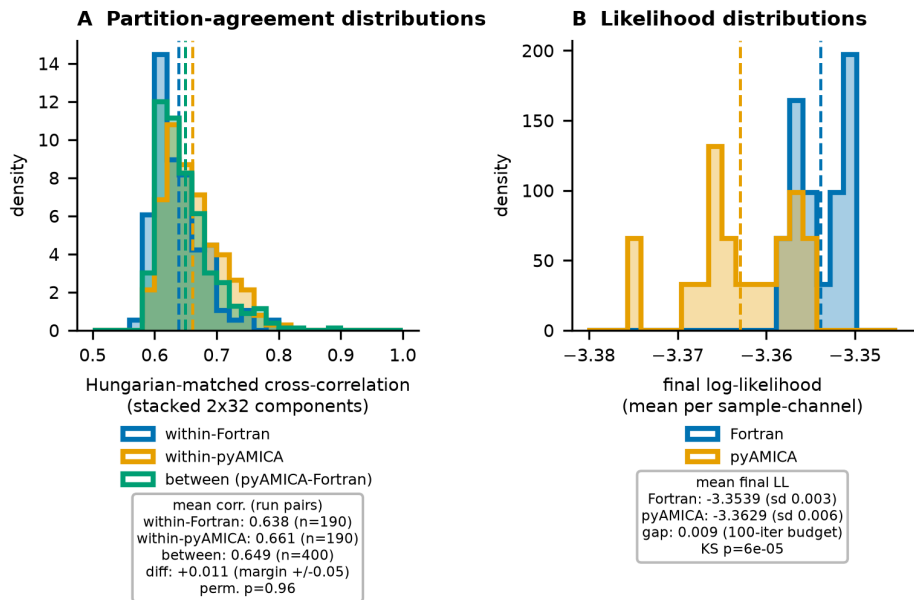


Figure 1: Multi-model solution-ensemble partition-correlation distributions (panel A) and log-likelihood distributions (panel B) for 20 pacica and 20 Fortran fits of the sample EEG; dashed lines mark each distribution's mean. The within-Fortran, within-pamica, and between-implementation correlation distributions overlap, so the single-run correlation reflects the estimator's intrinsic run-to-run spread rather than a gap to the reference. Panel B's apparent separation is a 0.009 log-likelihood gap on a ~ 0.035 axis.

77 All backends converge to the same single-model log-likelihood on real EEG (maximum pairwise
78 difference ~ 0.003). On Apple Silicon, MLX is the fastest backend and flat with channel count
79 (Table 2); PyTorch-MPS is never a win. Double-precision CUDA is the reproducible NVIDIA
80 path; native Fortran scales with CPU cores and, with enough cores pinned, can beat the
81 GPU on a larger, hotter host, though it does not catch Apple's MLX on laptop hardware.
82 A data-size sweep (documentation) shows cross-backend component agreement rising with
83 frames per channel and plateauing near 0.98 once well-determined, where two independent
84 double-precision implementations agree at a mean of 0.995; single-precision runs agree with
85 double precision to four to five significant digits, so double precision stays the default for parity.

Table 2: Single-model throughput on real 70-channel EEG ($n_{\text{mix}}=3$, $\text{pdftype}=0$, $\text{block_size}=512$; warm, minimum of repeated runs). CPU, MPS, and MLX on Apple Silicon; CUDA on a separate NVIDIA RTX 4090 (float32 comparable, ~ 36 ms). The two native-Fortran rows are from a separate core-count sweep (documentation) at each backend's plateau; the other CPU rows use platform-default threads and are not core-matched to Fortran. Unlike the correctness comparison, this benchmark uses external data (OpenNeuro ds002718, one subject so far) and specific GPU hardware.

Backend (device)	Precision	ms / iteration
MLX (Apple GPU)	float32	25
CUDA (NVIDIA RTX 4090)	float64	39
Native Fortran (Intel Core i9-13900K, 24 cores)	float64	30
PyTorch CPU (Apple Silicon)	float64	193
Native Fortran (Apple Silicon, 8 cores)	float64	70
PyTorch MPS (Apple GPU)	float32	255
NumPy (reference, Apple Silicon)	float64	622

86 The correctness harness never uses synthetic data; the multi-model and score-function checks
 87 need no external download (bundled sample only). The full performance tables, per-run
 88 Amari-distance detail, data-size sweep, and step-by-step reproduction commands are in the
 89 documentation (<https://eeglab.org/pAMICA/guides/validation/>).

90 State of the field

91 `pamica` complements rather than replaces EEGLAB (Delorme & Makeig, 2004) and its Fortran
 92 AMICA plugin: it shares EEGLAB's output format, adds GPU support and a Python API, and
 93 can run the reference Fortran itself through a bundled dependency-free native build (no MKL
 94 or MPI runtime). Two other Python AMICA reimplementations have appeared (Esmaili, 2025;
 95 Herforth, 2026), both of which provide MNE-Python-compatible objects; `pamica` leads with a
 96 scikit-learn-style array API, byte-identical EEGLAB I/O, and an MLX backend for Apple GPUs,
 97 and also ships an MNE-Python wrapper (an optional extra). What distinguishes `pamica` is the
 98 rigor and scope of its Fortran-parity validation beyond what either alternative publishes (Table
 99 1).

100 Acknowledgements

101 We thank Jason Palmer and Ken Kreutz-Delgado, co-developers of AMICA, for the reference
 102 implementation, and the EEGLAB community for the tools and sample data used to validate
 103 this work. Two of the authors are original developers of the methods `pamica` builds on: S.M.
 104 co-developed the AMICA algorithm (Palmer et al., cited above) and A.D. is a lead developer of
 105 EEGLAB (Delorme & Makeig, 2004). This work was supported by The Swartz Foundation (Old
 106 Field, NY) to the Swartz Center for Computational Neuroscience and by National Institutes of
 107 Health grant R01-NS047293 (to A.D. and S.M.).

108 References

- 109 Ablin, P., Cardoso, J.-F., & Gramfort, A. (2018). Faster independent component analysis
 110 by preconditioning with hessian approximations. *IEEE Transactions on Signal Processing*,
 111 66(15), 4040–4053. <https://doi.org/10.1109/TSP.2018.2844203>
- 112 Amari, S.-I. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2),
 113 251–276. <https://doi.org/10.1162/089976698300017746>

- 114 Amari, S.-I., Cichocki, A., & Yang, H. H. (1996). A new learning algorithm for blind signal
115 separation. *Advances in Neural Information Processing Systems*, 8, 757–763.
- 116 Bell, A. J., & Sejnowski, T. J. (1995). An information-maximization approach to blind
117 separation and blind deconvolution. *Neural Computation*, 7(6), 1129–1159. <https://doi.org/10.1162/neco.1995.7.6.1129>
118
- 119 Delorme, A., & Makeig, S. (2004). EEGLAB: An open source toolbox for analysis of single-
120 trial EEG dynamics including independent component analysis. *Journal of Neuroscience*
121 *Methods*, 134(1), 9–21. <https://doi.org/10.1016/j.jneumeth.2003.10.009>
- 122 Delorme, A., Palmer, J., Onton, J., Oostenveld, R., & Makeig, S. (2012). Independent EEG
123 sources are dipolar. *PLoS ONE*, 7(2), e30135. [https://doi.org/10.1371/journal.pone.](https://doi.org/10.1371/journal.pone.0030135)
124 [0030135](https://doi.org/10.1371/journal.pone.0030135)
- 125 Esmaeili, S. (2025). *Amica: Native python implementation of AMICA for MNE-Python and*
126 *scientific EEG workflows*. <https://github.com/snesmaeili/amica>.
- 127 Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj,
128 R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data
129 analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267. <https://doi.org/10.3389/fnins.2013.00267>
130
- 131 Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible*
132 *machine learning on apple silicon*. <https://github.com/ml-explore/mlx>.
- 133 Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,
134 Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with
135 NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- 136 Herforth, J. (2026). *Pyamica: PyTorch AMICA, adaptive mixture independent component*
137 *analysis*. <https://github.com/DerAndereJohannes/pyamica>.
- 138 Hyvärinen, A., & Oja, E. (2000). Independent component analysis: Algorithms and applications.
139 *Neural Networks*, 13(4-5), 411–430. [https://doi.org/10.1016/S0893-6080\(00\)00026-5](https://doi.org/10.1016/S0893-6080(00)00026-5)
- 140 Lee, T.-W., Girolami, M., & Sejnowski, T. J. (1999). Independent component analysis using
141 an extended infomax algorithm for mixed subgaussian and supergaussian sources. *Neural*
142 *Computation*, 11(2), 417–441. <https://doi.org/10.1162/089976699300016719>
- 143 Palmer, J. A., Kreutz-Delgado, K., Rao, B. D., & Makeig, S. (2008). Newton method for the
144 ICA mixture model. *2008 IEEE International Conference on Acoustics, Speech and Signal*
145 *Processing (ICASSP)*, 1805–1808. <https://doi.org/10.1109/ICASSP.2008.4517982>
- 146 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,
147 Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-
148 performance deep learning library. *Advances in Neural Information Processing Systems*, 32,
149 8024–8035.
- 150 Pion-Tonachini, L., Kreutz-Delgado, K., & Makeig, S. (2019). ICLabel: An automated
151 electroencephalographic independent component classifier, dataset, and website.
152 *NeuroImage*, 198, 181–197. <https://doi.org/10.1016/j.neuroimage.2019.05.026>
- 153 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
154 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:
155 Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3),
156 261–272. <https://doi.org/10.1038/s41592-019-0686-2>